AI-Driven DNA Sequence Classification

Kartavya Mandora
Master in Artificial Intelligence
Long Island University
New York,USA
kartavyapravinsinh.mandora@my.liu.edu

Krupalkumar Patel
Master in Artificial Intelligence
Long Island University
New York,USA
krupalkumaralpeshkumar.patel@my.liu.edu

Swayam Shah
Master in Artificial Intelligence
Long Island University
New York,USA
swayamjigneshbhai.shah@my.liu.edu

***Abstract*— Accurate identification of splice junctions in DNA sequences is essential for understanding gene expression and genomic structure. This project proposes a hybrid deep learning model combining Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) for multi-class DNA sequence classification. The model classifies sequences into Exon-Intron (EI), Intron-Exon (IE), and Non-splice (N) categories. DNA sequences are represented using binary-encoded features corresponding to nucleotide positions to preserve positional information. The CNN component extracts local sequence patterns, while the BiLSTM captures long-range dependencies and contextual relationships. The proposed model is evaluated using accuracy, precision, recall, and F1-score, and compared with baseline models such as Logistic Regression, Support Vector Machine, and Random Forest. Results demonstrate that the hybrid CNN-BiLSTM approach improves classification performance and provides a reliable tool for splice junction prediction in genomic analysis.**

Keywords*— *DNA Sequence Classification, Splice Junction Prediction, Convolutional Neural Network (CNN), Bidirectional Long Short-Term Memory (BiLSTM), Deep Learning, Bioinformatics, Multi-class Classification, One-Hot Encoding

I. Introduction

DNA sequence classification is a fundamental task in bioinformatics, playing a critical role in understanding gene structure and function. One of the most important problems in this domain is the identification of splice junctions, which define the boundaries between exons and introns in eukaryotic DNA. Accurate detection of these splice sites is essential for proper gene expression analysis, as errors in splicing can lead to serious genetic disorders and diseases. Traditional computational methods for splice site prediction often rely on manually engineered features and classical machine learning models, which may fail to capture complex patterns present in biological sequences.

This approach aims to provide a more accurate and efficient tool for splice junction prediction, supporting genomic research and improving the reliability of computational gene analysis.

II. Practical Applications

A. Applications of DNA Sequence Classification System

- **Splice Junction Detection**
  - *Automated Detection*: Detects exon-intron (EI) and intron-exon (IE) boundaries using deep learning models.
  - *Improved Accuracy:* Reduces errors compared to traditional machine learning methods.
- **Gene Structure Analysis**
  - *Genome Annotation: Helps identify gene structures and boundaries within DNA sequences.*
  - Understanding Gene Expression: Supports analysis of how genes are formed and processed.
- **Genomic Data Analysis**
  - *Large-scale Processing: Efficiently handles high-throughput sequencing data.*
  - *Pattern Recognition:* Learns complex biological patterns directly from raw DNA sequences.
- **Disease Analysis and Genetic Disorder Detection**
  - *Splicing Error Identification:* Detects abnormal splice junctions that may lead to genetic disorders
  - *Research Support:* Helps researchers study the relationship between splicing patterns and diseases such as cancer.

B. Maintaining the Integrity of the Specifications

Maintaining the integrity of the specifications is essential when implementing deep learning models for DNA sequence classification. By using a hybrid CNN + BiLSTM architecture in frameworks such as TensorFlow or PyTorch, the model adheres to a well-defined structure that captures both local sequence patterns and long-range dependencies. Ensuring that data preprocessing steps, including sequence validation, fixed-length window selection, and one-hot encoding, are consistently applied is critical for preserving input quality and biological relevance.

It is also important to carefully tune hyperparameters such as learning rate, number of filters, LSTM units, batch size, and epochs to avoid overfitting and underfitting. Techniques such as dropout, early stopping, and stratified data splitting help maintain model stability and generalization. Additionally, evaluating the model using appropriate multi-class metrics such as accuracy, precision, recall, and F1-score ensures that performance aligns with project objectives. By following these best practices, the system maintains reliability, consistency, and accuracy in splice junction classification.

III. RELATED WORK

DNA sequence classification has been widely studied in the field of bioinformatics, with various traditional and modern approaches proposed over the years. Earlier methods mainly relied on statistical and rule-based techniques, such as sequence alignment tools and pattern matching algorithms. Techniques like BLAST (Basic Local Alignment Search Tool) were commonly used to compare DNA sequences and identify similarities. While these methods are effective, they are often time-consuming and computationally expensive when dealing with large-scale genomic data.

With the advancement of machine learning, researchers started applying classification algorithms such as Support Vector Machines (SVM), Decision Trees, and Naïve Bayes

for DNA sequence classification. These models require manual feature extraction techniques like k-mer representation, which converts DNA sequences into numerical form. Although these methods improved performance compared to traditional approaches, they still depend heavily on feature engineering and may not capture complex patterns in biological data.

More recently, deep learning approaches have gained significant attention due to their ability to automatically extract features from raw data. Models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), including Long Short-Term Memory (LSTM) networks, have been widely used for DNA sequence analysis. CNNs are effective in identifying local patterns within sequences, while RNNs are useful for capturing sequential dependencies. These methods have shown improved accuracy and efficiency compared to traditional machine learning techniques.

Despite these advancements, there are still some challenges in DNA sequence classification, such as handling large datasets, avoiding overfitting, and ensuring model interpretability. In this project, an AI-driven approach is used to classify DNA sequences efficiently by leveraging modern machine learning/deep learning techniques, aiming to achieve better accuracy and performance compared to existing methods.

IV. Methodology

The methodology of this project focuses on building an AI-driven system for accurate DNA sequence classification. The overall process includes data collection, preprocessing, feature extraction, model training, and evaluation.

A. Dataset Description

The dataset used in this project consists of DNA sequences, where each sequence is composed of nucleotide bases such as Adenine (A), Thymine (T), Cytosine (C), and Guanine (G). Each sequence is associated with a specific class label, which represents its biological category. The dataset is divided into training and testing sets to evaluate the performance of the model.
The dataset contains labelled DNA sequences categorized into EI, IE, and N classes, with each sequence having a fixed length used for model training.

B. Data Preprocessing

Before training the model, the raw DNA sequences are preprocessed to make them suitable for machine learning. This includes cleaning the data, removing any invalid or missing values, and ensuring consistency in sequence length if required. Since machine learning models cannot directly process string data, the DNA sequences are converted into numerical form using encoding techniques.

C. Feature Extraction / Encoding

To transform DNA sequences into numerical representations, encoding methods such as k-mer representation or one-hot encoding are used. In this approach, each nucleotide (A, T, C, G) is converted into a numerical format that can be understood by the model. This step is important because it helps the model learn patterns and relationships within the DNA sequences.

D. Model Selection

In this project, a hybrid CNN-BiLSTM model is used for DNA sequence classification. The CNN component extracts local features from DNA sequences, while the BiLSTM captures sequential dependencies, resulting in improved classification performance.

E. Training Process

The dataset is split into training and testing sets. The model is trained using the training data, where it learns the relationship between input sequences and their corresponding labels. During training, optimization techniques are applied to minimize the loss function and improve accuracy. Hyperparameters such as learning rate, number of epochs, and batch size are adjusted to achieve better performance.

F. Model Evaluation

After training, the model is evaluated using the test dataset. Performance metrics such as accuracy are used to measure how well the model classifies unseen DNA sequences. The evaluation helps in understanding the effectiveness and reliability of the proposed system.

V. System Architecture / Model Design

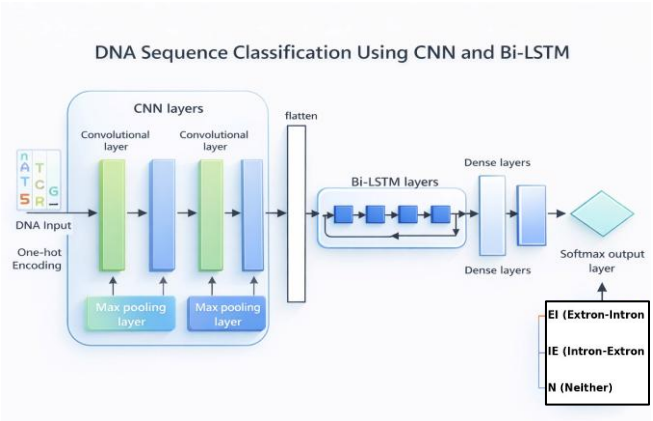

Figure 1: Model Architecture

A. Convolutional Neural Network (CNN) Architecture

The Convolutional Neural Network (CNN) is used as the first component of the proposed model to extract local features from DNA sequences. Initially, the input DNA sequences are converted into numerical form using one-hot encoding, where each nucleotide (A, T, C, G) is represented as a binary vector. This encoded input is then fed into the CNN layers.

The CNN architecture consists of multiple convolutional layers followed by max-pooling layers. The convolutional layers apply filters to the input data to capture important local patterns and motifs within the DNA sequences. These filters help in identifying meaningful biological features such as recurring nucleotide patterns.

After each convolutional operation, a max-pooling layer is applied to reduce the dimensionality of the feature maps while retaining the most important information. This helps in reducing computational complexity and prevents overfitting.

The output of the CNN layers is then flattened into a one-dimensional vector, which serves as input to the next stage of the model. Overall, the CNN component plays a crucial role in extracting spatial features from DNA sequences effectively.

B. Bidirectional Long Short-Term Memory (Bi-LSTM)

The Bidirectional Long Short-Term Memory (Bi-LSTM) network is used after the CNN layers to capture sequential dependencies in the DNA data. Unlike traditional neural networks, LSTM is specifically designed to handle sequential data and retain information over long sequences.

In this model, a bidirectional LSTM is used, which processes the sequence in both forward and backward directions. This allows the model to capture context from both past and future positions in the DNA sequence, leading to a better understanding of the relationships between nucleotides.

The flattened output from the CNN layers is provided as input to the Bi-LSTM layers. These layers learn long-term dependencies and sequential patterns that are not captured by CNN alone. This combination of CNN and Bi-LSTM improves the overall feature representation of the data.

After processing through the Bi-LSTM layers, the output is passed to fully connected (dense) layers, which perform the final classification. A softmax activation function is used in the output layer to classify the DNA sequences into different categories such as EI, IE, and N splice junction categories.

VI. Implementation

A. Software and Tools Used

The proposed model is implemented using Python with TensorFlow and Keras for deep learning. Additional libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn are used for data preprocessing and evaluation. The model is executed on CPU by disabling GPU usage.

B. Data Preparation and Splitting

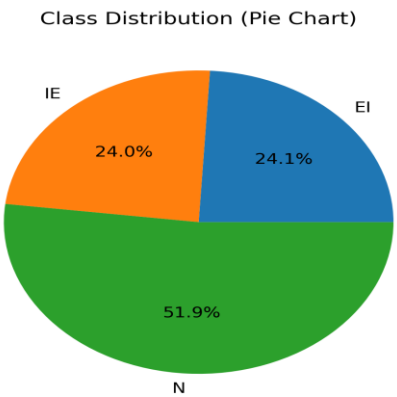

Figure 2: Dataset Class Distribution

The DNA dataset is loaded from a CSV file and preprocessed to separate input features and class labels. The labels are encoded using Label Encoder and converted into one-hot format. The input data is reshaped into (60, 3) to match the model requirements. The dataset is split into training (80%), validation (10%), and testing (10%) sets using a random state of 42. Class weights are computed to handle class imbalance.

C. Model Training Setup

The CNN-BiLSTM model is trained using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy loss. The training is performed for a maximum of **50 epochs** with a **batch size of 32**. Early Stopping (patience = 8), Reduce LR On Plateau, and Model Checkpoint are used to improve training performance and prevent overfitting.

The CNN-BiLSTM model was optimized using carefully selected hyperparameters to ensure efficient learning and good generalization. The model was trained using the Adam optimizer with a learning rate of 0.001 and a batch size of 32 for up to 50 epochs, with early stopping applied to prevent overfitting. The convolutional layers used 64 and 128 filters with kernel sizes of 3 and 5 to effectively captures both local nucleotide patterns and long-range sequence dependencies, while the BiLSTM layer with 64 units modeled long-range dependencies. A dropout rate of 0.3 was applied to improve generalization. The architecture includes two dense layers: a hidden layer with 64 neurons for feature learning and an output layer with softmax activation for multi-class classification. The model was trained using categorical cross-entropy loss, appropriate for splice junction prediction tasks.

D. Evaluation and Baseline Comparison

The model is evaluated on the test dataset using accuracy, precision, recall, and F1-score. A confusion matrix and classification report are generated for detailed analysis. Additionally, baseline models such as Logistic Regression, SVM, and Random Forest (200 estimators) are implemented for performance comparison.

VII. Results and Discussion

The performance of the proposed CNN-BiLSTM model was evaluated on the test dataset using multiple evaluation metrics, including accuracy, precision, recall, and F1-score. The model was trained for a maximum of 50 epochs with a batch size of 32, and early stopping was applied to prevent overfitting.

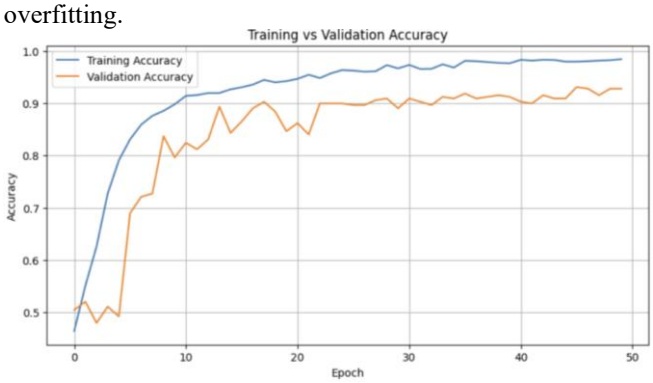

Figure 3: Training vs Validation Accuracy

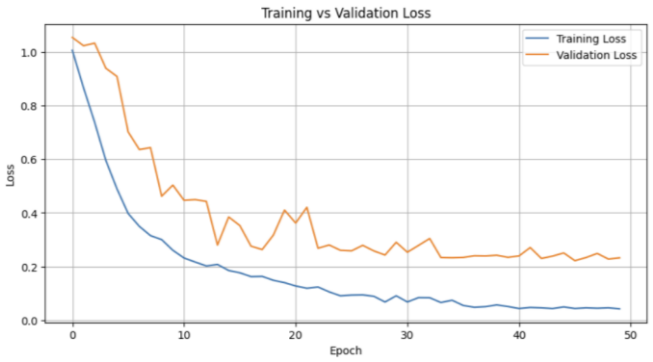

Figure 4: Training vs Validation Loss

The results show that the model is able to effectively classify DNA sequences into their respective classes. The convolutional layers successfully extract local patterns from the sequences, while the bidirectional LSTM captures long-term dependencies, resulting in improved classification performance.

The proposed CNN-BiLSTM model demonstrates strong performance in DNA sequence classification.

The performance of the proposed CNN-BiLSTM model was evaluated using standard classification metrics. The model achieved a **training accuracy of 98.43%** and **a testing accuracy of 94.36%**, indicating good generalization performance. Additionally, the model obtained **a precision of 0.93**, **recall of 0.95**, and an **F1-score of 0.94**, demonstrating balanced and reliable classification across all classes.

Comparison with Baseline Models

Metric	Value
Accuracy	94.36%
Precision (Macro)	93.12%
Recall (Macro)	94.73%
F1 Score (Macro)	93.85%

The ROC curves demonstrate excellent discriminative capability, with AUC values of 1.00, 0.99, and 0.98 for the three classes, confirming the model's ability to distinguish between splice junction categories with high confidence. Similarly, the Precision–Recall curves show high average precision values, particularly for the EI and N classes, highlighting the model's robustness in handling class imbalance and correctly identifying splice boundaries. The classification report further indicates consistent performance across all classes, with particularly high precision for the non-splice class and strong recall for splice junction classes. Overall, these results validate that the hybrid CNN-BiLSTM architecture effectively captures both local nucleotide patterns and long-range dependencies, leading to improved generalization and reliable splice junction prediction. The confusion matrix indicates that the CNN-BiLSTM model achieves high classification accuracy, with most predictions correctly aligned along the diagonal, demonstrating strong performance across all classes. Minor misclassifications are observed primarily between the EI and IE classes, which is expected due to their biological similarity, while the non-splice class (N) is predicted with the highest precision.

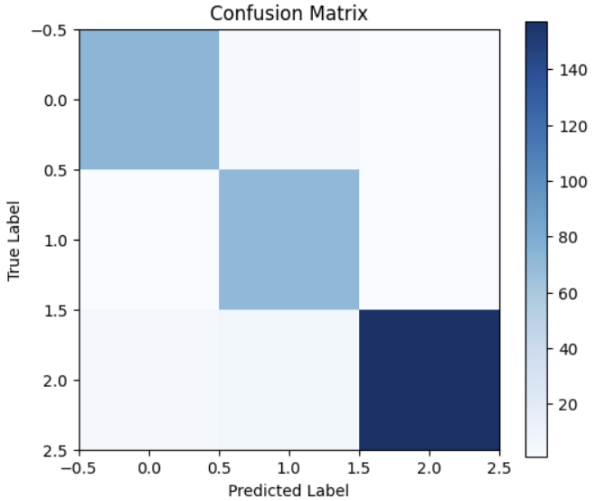

Figure 5: Confusion Matrix of CNN-BiLSTM Model

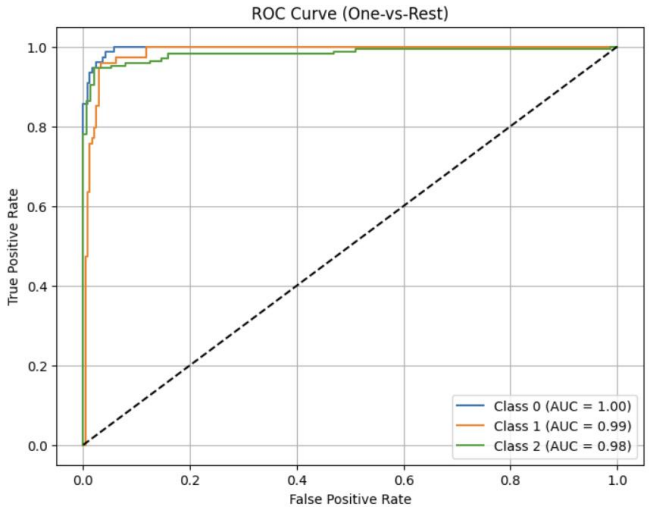

Figure 6: ROC curve showing model discrimination performance

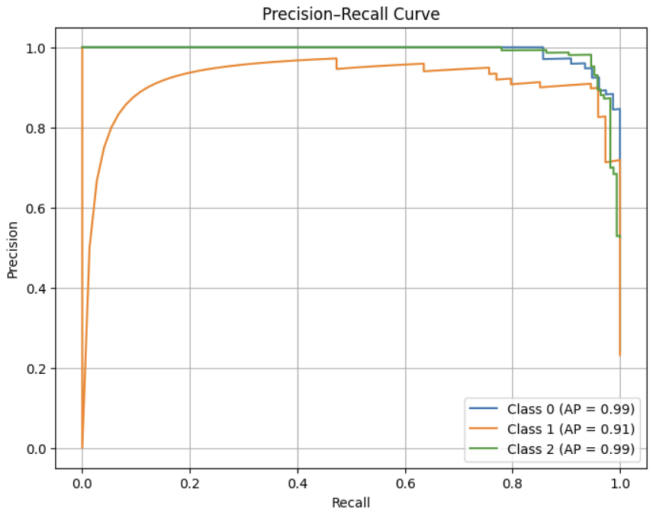

Figure 7: Precision-Recall curve

Additionally, techniques such as dropout, class weighting, and learning rate adjustment were applied to improve generalization and handle class imbalance. These strategies helped in reducing overfitting and improving the robustness of the model.

VIII. Conclusion & Future Work

The proposed CNN-BiLSTM model demonstrates strong performance in splice junction classification, achieving a training accuracy of 98.43% and a testing accuracy of 94.36%, indicating effective generalization with minimal overfitting. The integration of convolutional layers and bidirectional LSTM enables the model to capture both local nucleotide patterns and long-range dependencies, resulting in balanced and reliable classification across EI, IE, and N classes. Comprehensive evaluation using accuracy, precision, recall, F1-score, ROC curves, and Precision-Recall curves confirms the robustness and high discriminative capability of the model for DNA sequence analysis.

Despite the strong performance achieved, further improvements can be explored to enhance the model's effectiveness and scalability. Future work will focus on optimizing hyperparameters and experimenting with additional machine learning and deep learning models such as Support Vector Machine (SVM), Random Forest, Logistic Regression, and Transformer-based architectures to improve sequence learning capabilities. Additionally, applying the model to larger and more diverse genomic datasets, along with integrating explainable AI techniques, can further improve model interpretability and real-world applicability in genomic analysis.

IX. ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to their professor, **Dr. Reda Nacif Elalaoui**, for his valuable guidance, continuous support, and insightful feedback throughout the course of this project. The authors also acknowledge the contributions and collaboration of all team members whose efforts made this work possible.
Additionally, the authors would like to thank the providers of the datasets and research resources that supported this study. Finally, the authors express their appreciation to their families and friends for their constant encouragement and support.

X. REFERENCES

[1] J. Zou, M. Huss, A. Abid, P. Mohammadi, A. Torkamani, and A. Telenti,
" A primer on deep learning in genomics," Nature Genetics, vol. 51, no. 1, pp. 12–18, 2019.

[2] Y. LeCun, Y. Bengio, and G. Hinton,
"Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[3] T. N. Kipf and M. Welling,
"Semi-supervised classification with graph convolutional networks,"
International Conference on Learning Representations (ICLR), 2017.

[4] D. R. Kelley,
"Predicting DNA sequence function with deep learning," Nature Methods, vol. 17, pp. 111–118, 2020.